

Aula 06 – Circuitos Combinatórios

Prof. João Fernando Mari

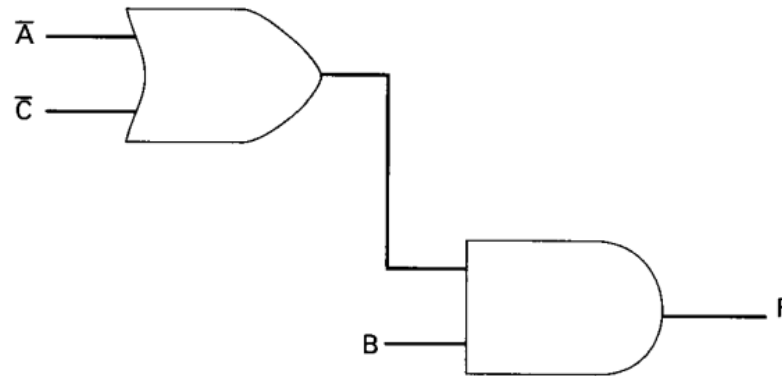
joaofmari.github.io

Roteiro

- Simplificação algébrica (sem Mapas de Karnaugh)
- Projeto de circuitos lógicos
- Universalidade das portas lógicas: AND, OR, NOT / AND, NOT / OR, NOT
- Universalidade das portas NAND e NOR
- Exemplo: Universalidade das portas NAND e NOR
- Multiplexadores (MUX)
- Decodificadores
- Exemplo: Decodificação de Endereços
- Memórias apenas de leitura (Memória ROM)
- Adição binária
- Adição binária – Usando XOR
- Somador de 4 bits
- “Vai um” antecipado (*carry lookahead*)
- Somador de 32 bits usando somadores de 8 bits

Simplificação algébrica (sem Mapas de Karnaugh)

- Aplicar as identidades da álgebra booleana para gerar uma função equivalente com menos variáveis
 - Apenas para expressões **mais simples**.
- Para expressões mais complexas
 - Utilizar os **Mapas de Karnaugh**
- EXEMPLO:
 - $F = A'B + BC' \rightarrow 5$ operadores
 - Simplificada: $F = B(A' + C') \rightarrow 4$ operadores



Projeto de circuitos lógicos

- Geralmente não utilizamos todos os tipos de portas lógicas em uma implementação de circuito lógico.
 - Um número menor de portas lógicas torna o **projeto mais simples**.
- Conjunto de portas lógicas **funcionalmente completos**:
 - Qualquer função booleana pode ser implementada usando apenas as portas de um conjunto
 - **AND, OR, NOT**
 - AND, NOT
 - OR, NOT
 - NAND
 - NOR

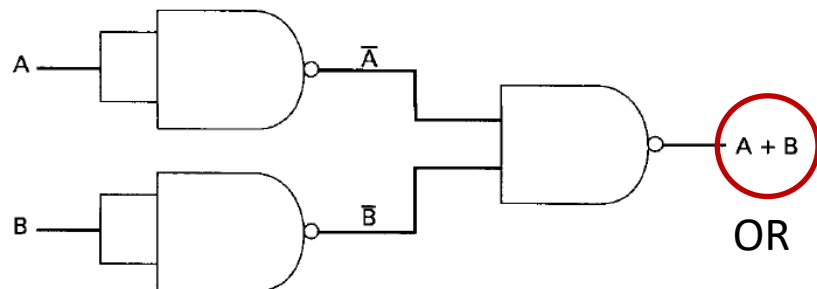
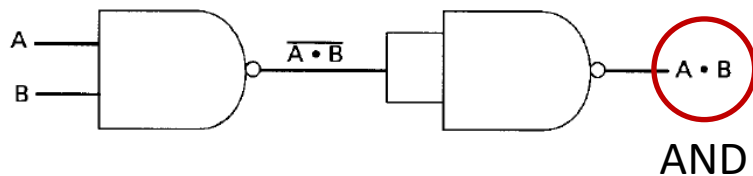
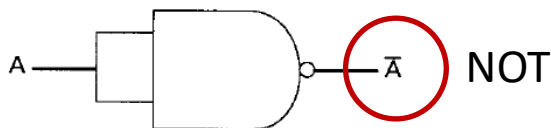
Universalidade das portas lógicas: AND, OR, NOT / AND, NOT / OR, NOT

- **AND, OR, NOT:**
 - Conjunto funcionalmente completo pois representam as três operações básicas da álgebra booleana.
- **AND, NOT:**
 - Para ser funcionalmente completo é necessário representar a função OR usando AND e NOT
 - Pelas leis de De Morgan: $A + B = (A' \cdot B')'$
 - $A \text{ OR } B = \text{NOT} ((\text{NOT } A) \text{ AND } (\text{NOT } B))$
- **OR, NOT:**
 - Analogamente ao conjunto AND, NOT
 - Pelas leis de De Morgan: $A \cdot B = (A' + B')'$
 - $A \text{ AND } B = \text{NOT} ((\text{NOT } A) \text{ OR } (\text{NOT } B))$

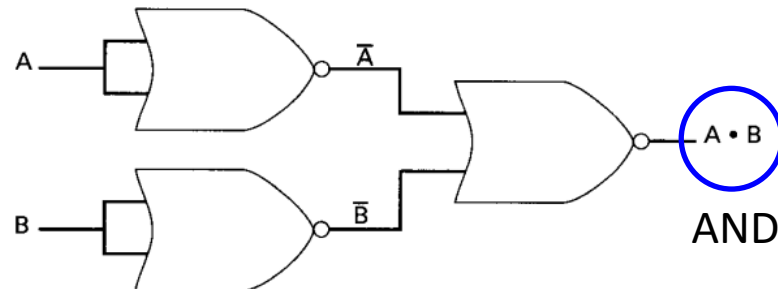
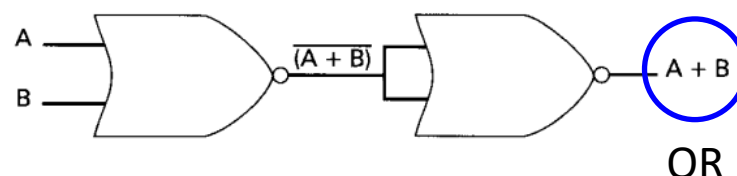
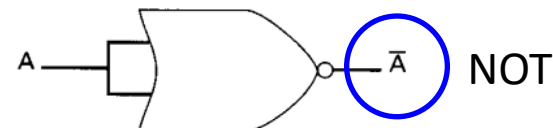
Universalidade das portas NAND e NOR

- As funções **AND**, **OR** e **NOT** podem ser implementadas usando apenas a porta **NAND** ou apenas a porta **NOR**.

NAND



NOR



[EXEMPLO] Universalidade das portas NAND e NOR

$$F = B(\overline{A} + \overline{C})$$

$$F = B(\overline{A} + \overline{C}) = (\overline{A}B) + (B\overline{C})$$

Por De Morgan:

$$F = \overline{\overline{\overline{A}B}} \bullet \overline{\overline{B\overline{C}}}$$

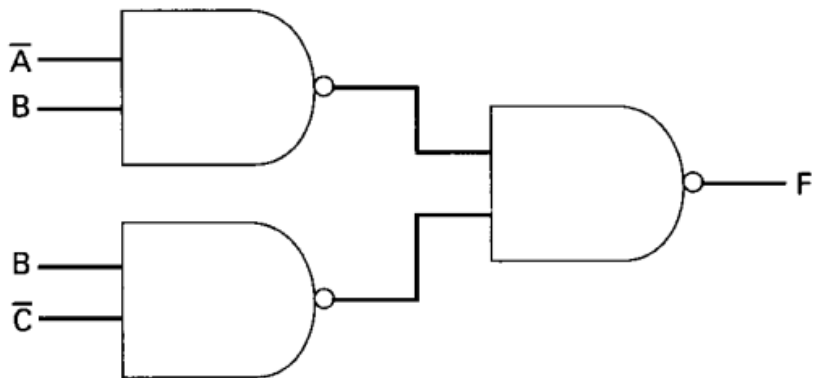
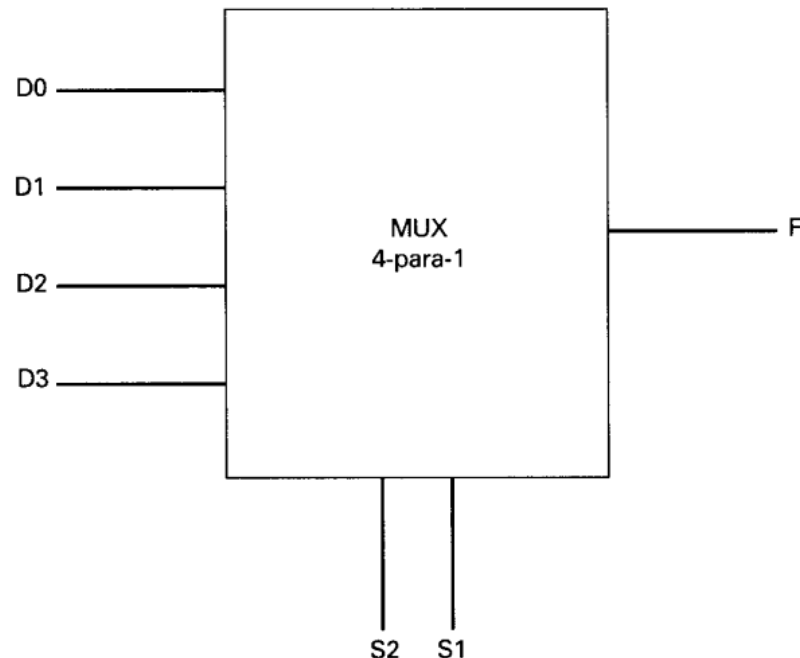


Tabela Verdade				
#	A	B	C	F
0	0	0	0	0
1	0	0	1	0
2	0	1	0	1
3	0	1	1	1
4	1	0	0	0
5	1	0	1	0
6	1	1	0	1
7	1	1	1	0

Multiplexadores (MUX)

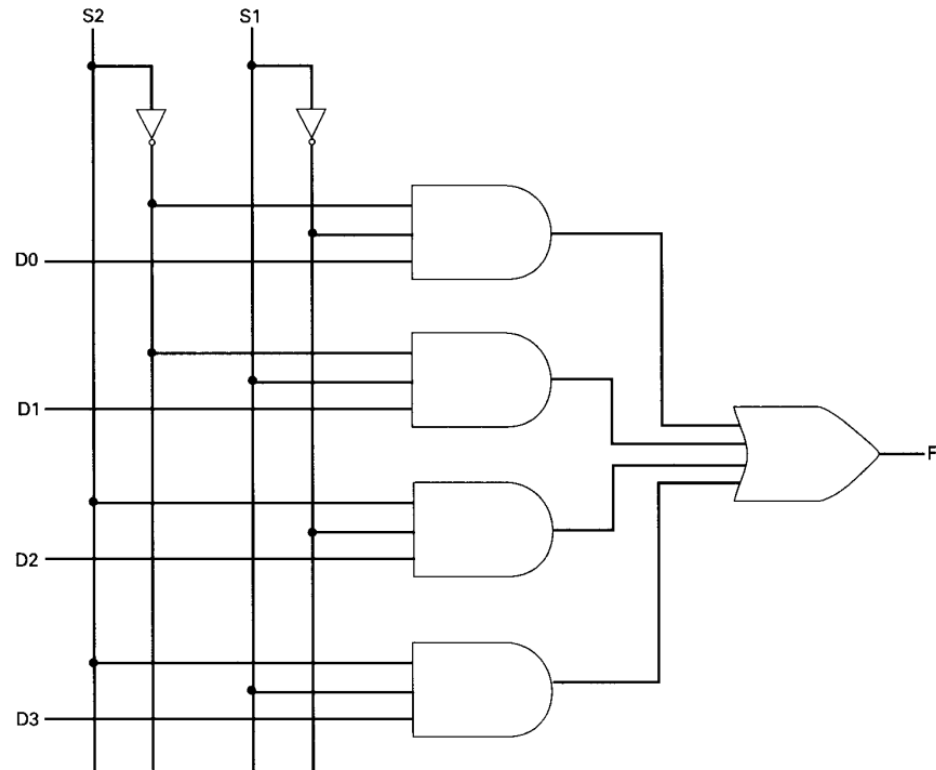
- Várias entradas.
- Uma única saída.
- A cada instante uma única entrada passa para a saída.
- Determinado por um conjunto de linhas de seleção.
- Para n linhas de controle:
 - 2^n entradas



Multiplexadores (MUX)

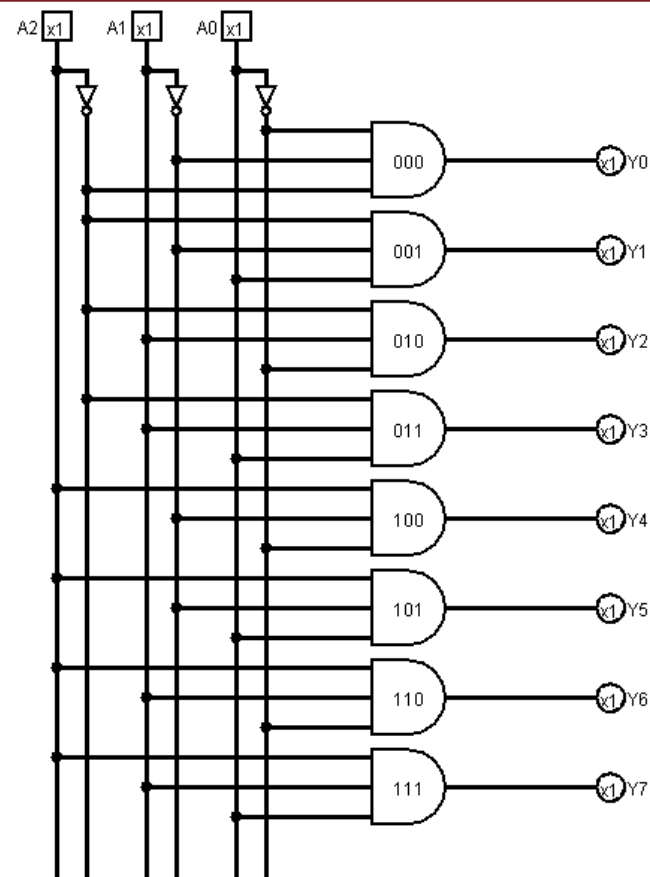
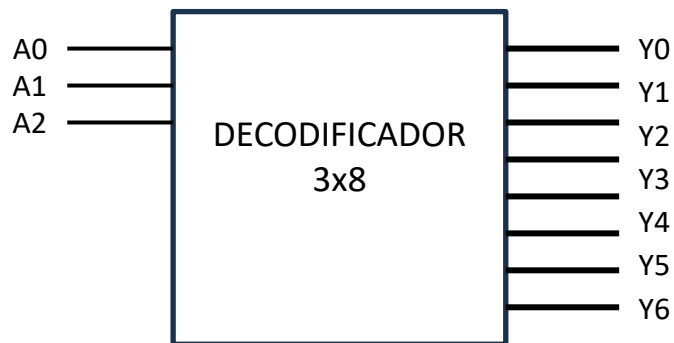
Tabela verdade do multiplexador 4-para-1

#	S2	S1	F
0	0	0	D0
1	0	1	D1
2	1	0	D2
3	1	1	D3



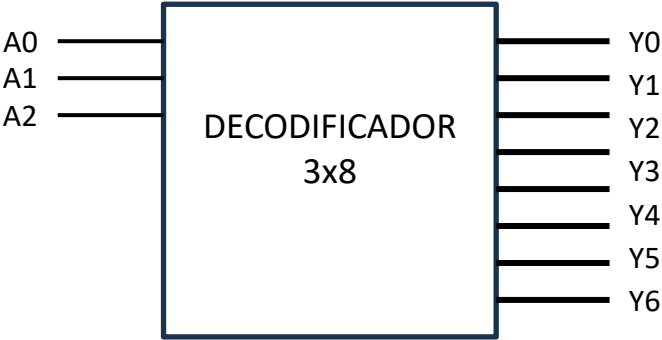
Decodificadores

- Circuito combinatório:
 - Um certo número de linhas de saída.
 - Apenas uma é ativada em cada instante,
 - Dependendo do padrão de sinais de entrada.

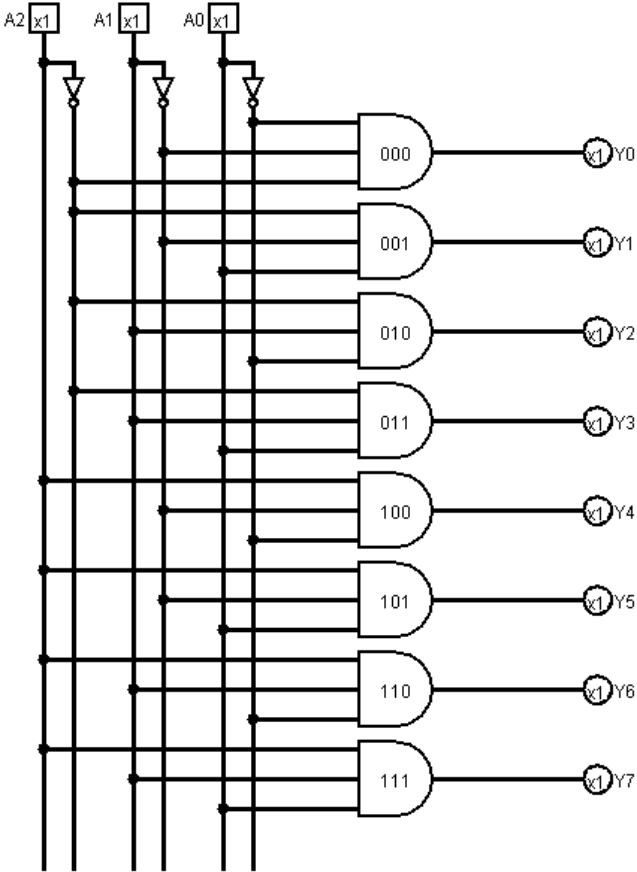


Decodificador com 3 entradas e $2^3 = 8$ saídas.

Decodificadores



#	Entradas			Saídas							
	A2	A1	A0	Y7	Y6	Y5	Y4	Y3	Y2	Y1	Y0
0	0	0	0	0	0	0	0	0	0	0	1
1	0	0	1	0	0	0	0	0	0	1	0
2	0	1	0	0	0	0	0	0	1	0	0
3	0	1	1	0	0	0	0	1	0	0	0
4	1	0	0	0	0	0	1	0	0	0	0
5	1	0	1	0	0	1	0	0	0	0	0
6	1	1	0	0	1	0	0	0	0	0	0
7	1	1	1	1	0	0	0	0	0	0	0



[EXEMPLO] Decodificação de Endereços

- Memória de 1 Kbyte
 - 4 pastilhas de memória RAM de 256 bytes (256 palavras de 8 bits)

#	Endereço (HEX)	Endereço (DEC)	Pastilha
0	0000-00FF	0 - 255	0
1	0100-01FF	256 - 511	1
2	0200-02FF	512 - 767	2
3	0300-03FF	768 - 1023	3

- Endereços de 10 bits
 - 8 bits menos significativos ($2^8 = 256$ palavras)
 - 2 bits mais significativos ($2^2 = 4$ pastilhas)
 - Selecionar uma das quatro pastilhas (decodificador)

[EXEMPLO] Decodificação de Endereços

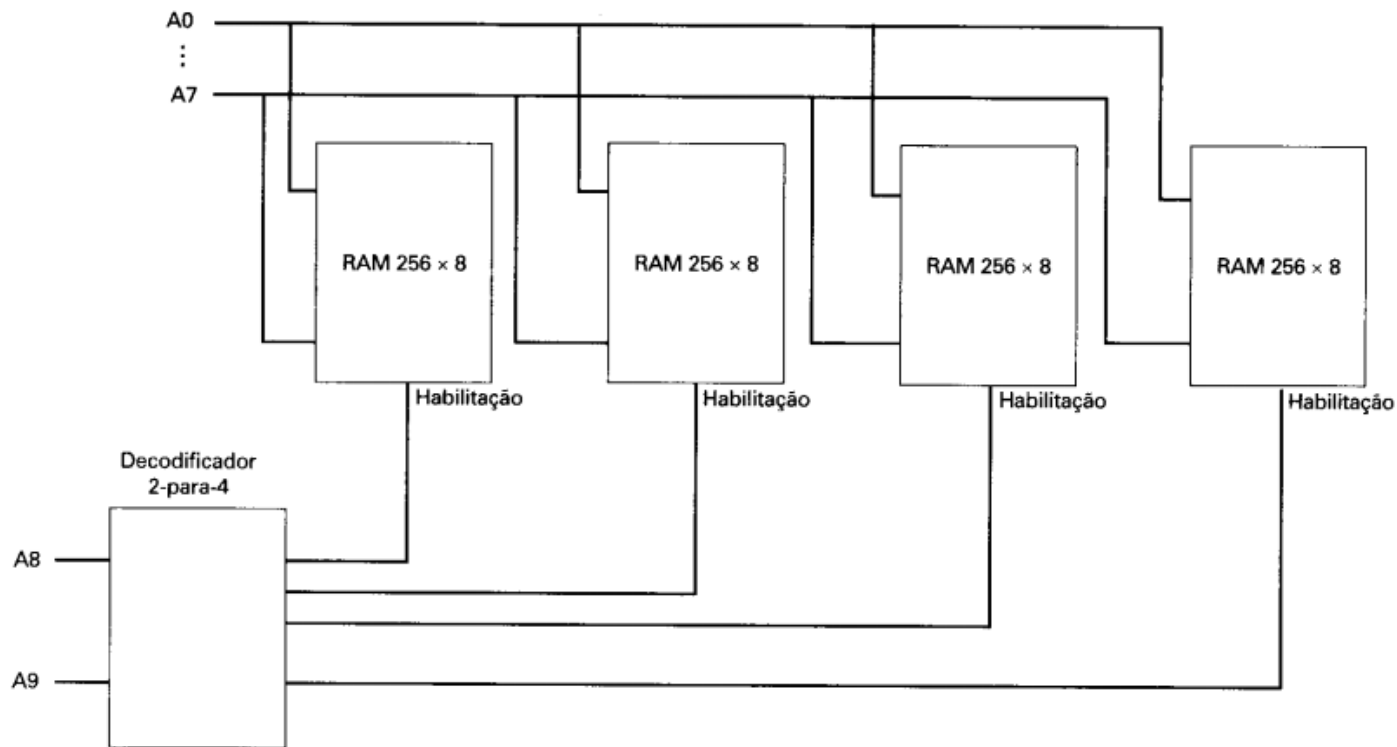
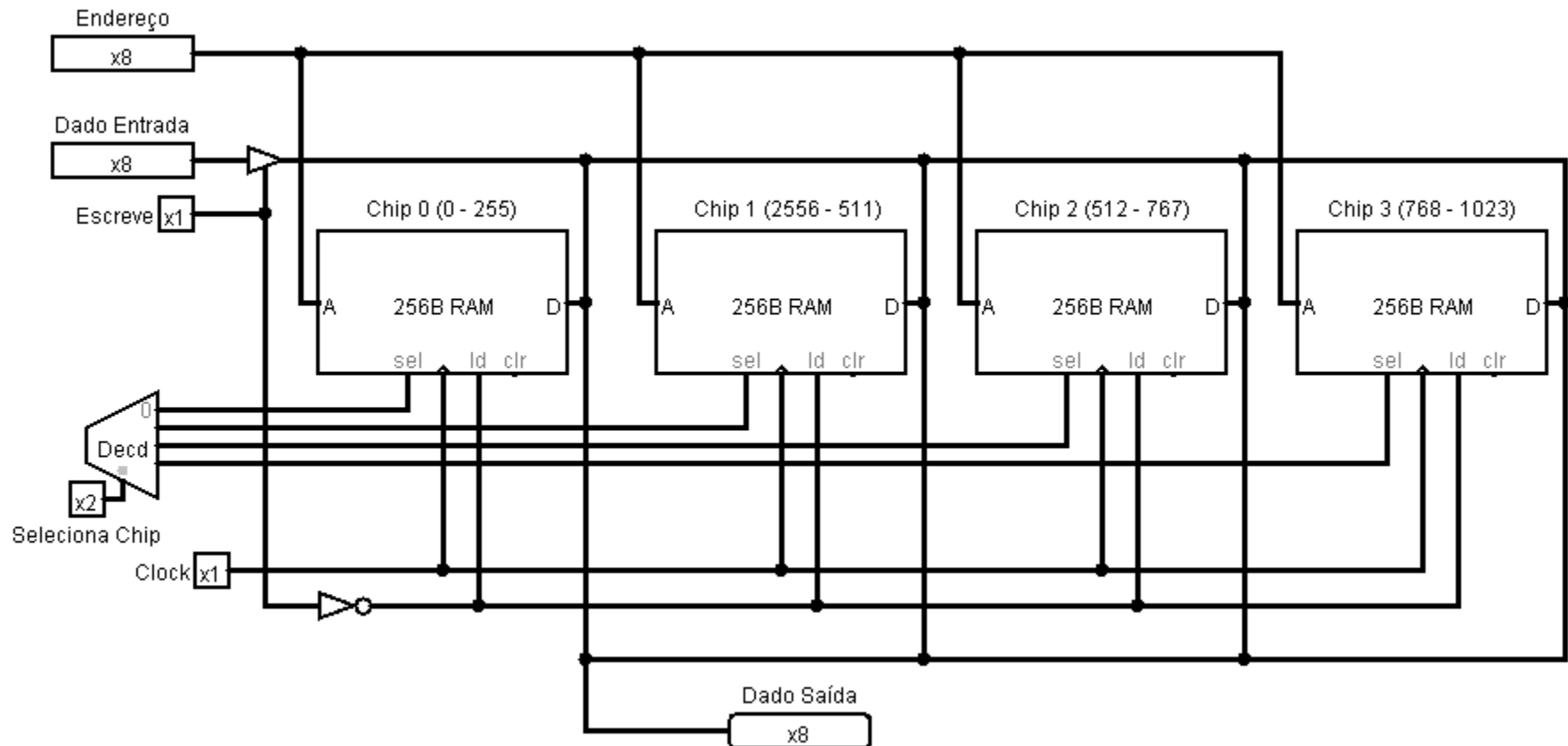


Figura A.16 Decodificação de endereços.

[EXEMPLO] Decodificação de Endereços (Logisim)



Memórias apenas de leitura (Memória ROM)

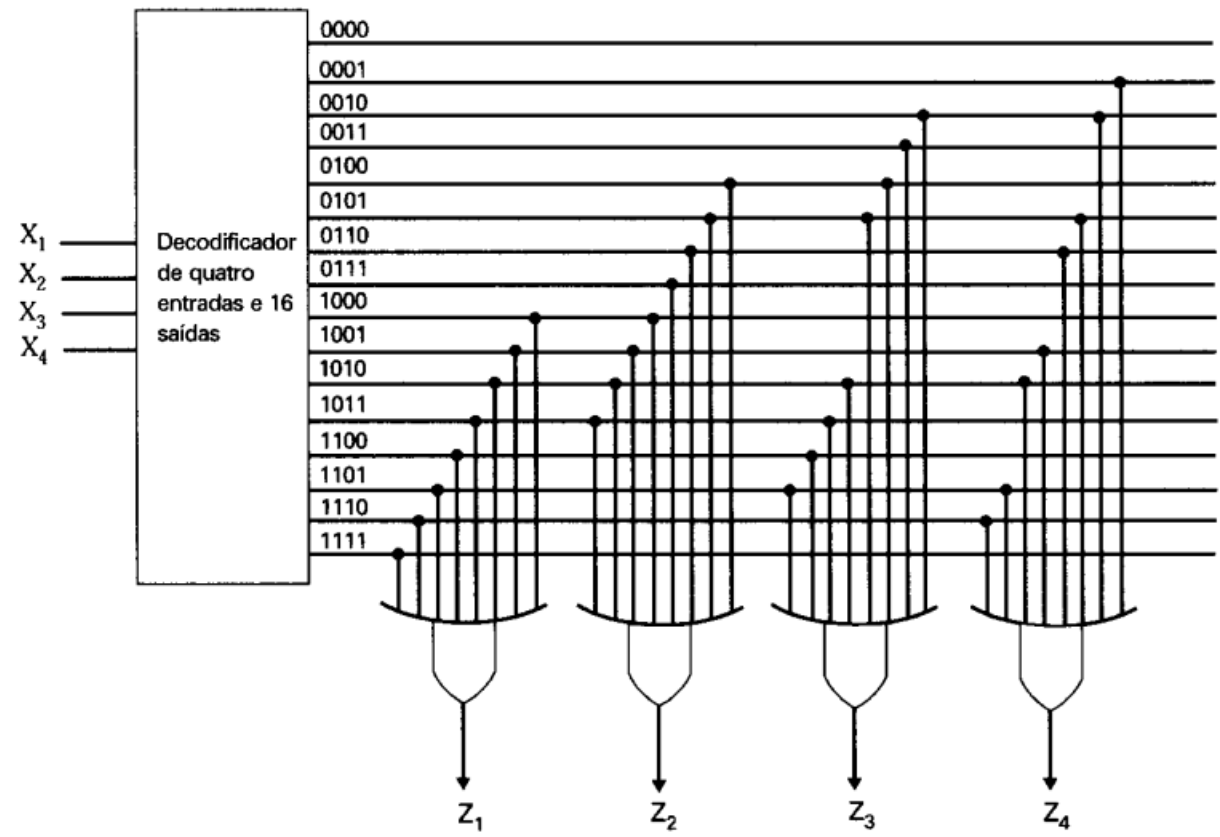
- Circuitos combinatórios:
 - São circuitos “sem memória”.
- ROM (Read-Only-Memory)
 - Memória implementada usando circuitos combinatórios.
 - Um **decodificador** e um **conjunto de portas OR**.
- Memória ROM de 64 bits
 - 16 palavras de 4 bits

Memórias apenas de leitura (Memória ROM)

- Entradas:
 - São as posições da memória ROM.
- Saídas:
 - Os dados que você quer armazenar.

	Entradas				Saídas			
#	X1	X2	X3	X4	Z1	Z2	Z3	Z4
0	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	1
2	0	0	1	0	0	0	1	1
3	0	0	1	1	0	0	1	0
4	0	1	0	0	0	1	1	0
5	0	1	0	1	0	1	1	1
6	0	1	1	0	0	1	0	1
7	0	1	1	1	0	1	0	0
8	1	0	0	0	1	1	0	0
9	1	0	0	1	1	1	0	1
10	1	0	1	0	1	1	1	1
11	1	0	1	1	1	1	1	0
12	1	1	0	0	1	0	1	0
13	1	1	0	1	1	0	1	1
14	1	1	1	0	1	0	0	1
15	1	1	1	1	1	0	0	0

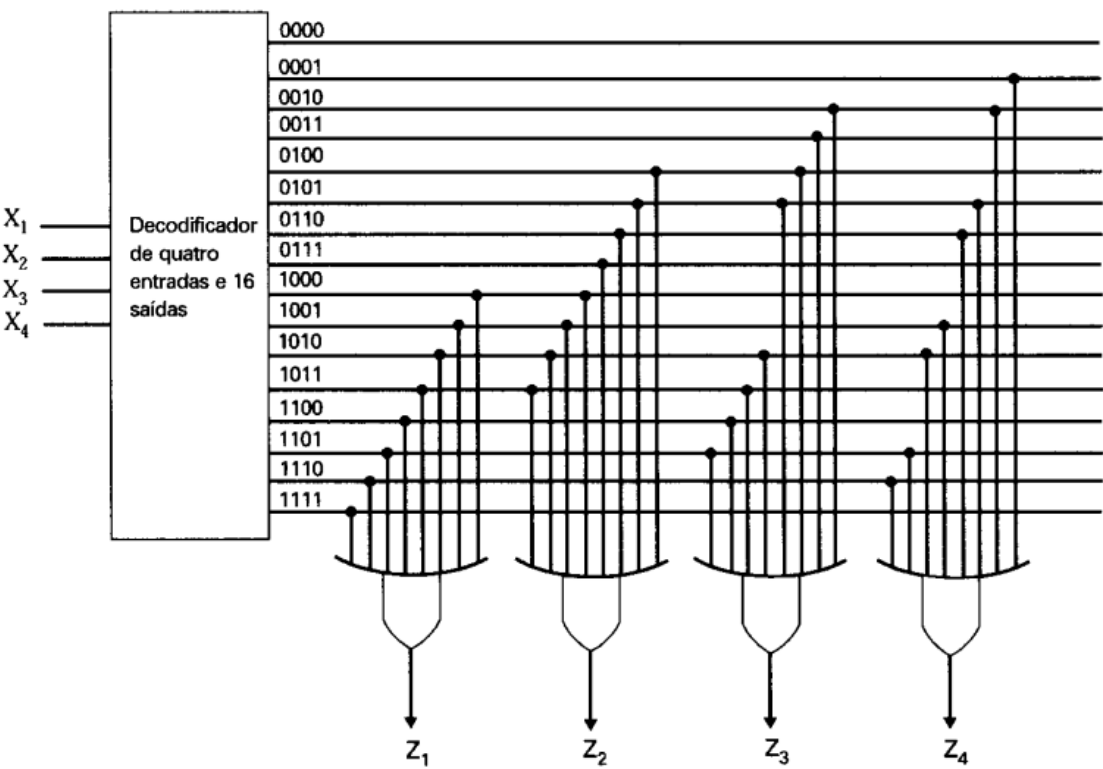
ROM de 64 bits



ROM de 64 bits

Tabela verdade para uma ROM.

Entradas					Saídas			
#	X1	X2	X3	X4	Z1	Z2	Z3	Z4
0	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	1
2	0	0	1	0	0	0	1	1
3	0	0	1	1	0	0	1	0
4	0	1	0	0	0	1	1	0
5	0	1	0	1	0	1	1	1
6	0	1	1	0	0	1	0	1
7	0	1	1	1	0	1	0	0
8	1	0	0	0	1	1	0	0
9	1	0	0	1	1	1	0	1
10	1	0	1	0	1	1	1	1
11	1	0	1	1	1	1	1	0
12	1	1	0	0	1	0	1	0
13	1	1	0	1	1	0	1	1
14	1	1	1	0	1	0	0	1
15	1	1	1	1	1	0	0	0



Circuitos Somadores

- Circuito combinatório
 - Operações aritmética
- Adição binária
 - Vai-um (Carry)

0	0	1	1
<u>+ 0</u>	<u>+ 1</u>	<u>+ 0</u>	<u>+ 1</u>
0	1	1	10

Circuitos Somadores

- Circuito combinatório
 - Operações aritmética
- Adição binária
 - Vai-um (Carry)

$$\begin{array}{r} 0 \\ + 0 \\ \hline 0 \end{array} \quad \begin{array}{r} 0 \\ + 1 \\ \hline 1 \end{array} \quad \begin{array}{r} 1 \\ + 0 \\ \hline 1 \end{array} \quad \begin{array}{r} 1 \\ + 1 \\ \hline 10 \end{array}$$

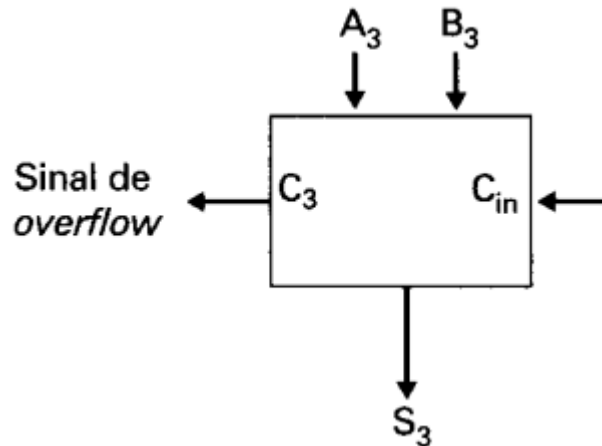
Adição de um único bit.

#	A	B	Soma	Vai-um
0	0	0	0	0
1	0	1	1	0
2	1	0	1	0
3	1	1	0	1

Adição binária

Adição de um único bit.

#	A	B	Soma	Vai-um
0	0	0	0	0
1	0	1	1	0
2	1	0	1	0
3	1	1	0	1



Adição com uma entrada de bit e 'vai-um'.

#	C_{in}	A	B	Soma	C_{out}
0	0	0	0	0	0
1	0	0	1	1	0
2	0	1	0	1	0
3	0	1	1	0	1
4	1	0	0	1	0
5	1	0	1	0	1
6	1	1	0	0	1
7	1	1	1	1	1

$$\text{Soma} = A'BC' + AB'C' + A'B'C + ABC$$

$$\text{Vai-um} = ABC' + A'BC + AB'C + ABC$$

$$\text{Obs.: } C = C_{in}.$$

Adição Binária

$$\text{Soma} = A'BC' + AB'C' + A'B'C + ABC$$

$$\text{Vai-um} = ABC' + A'BC + AB'C + ABC$$

Soma:

	BC			
	00	01	11	10
0		1		1
1	1		1	

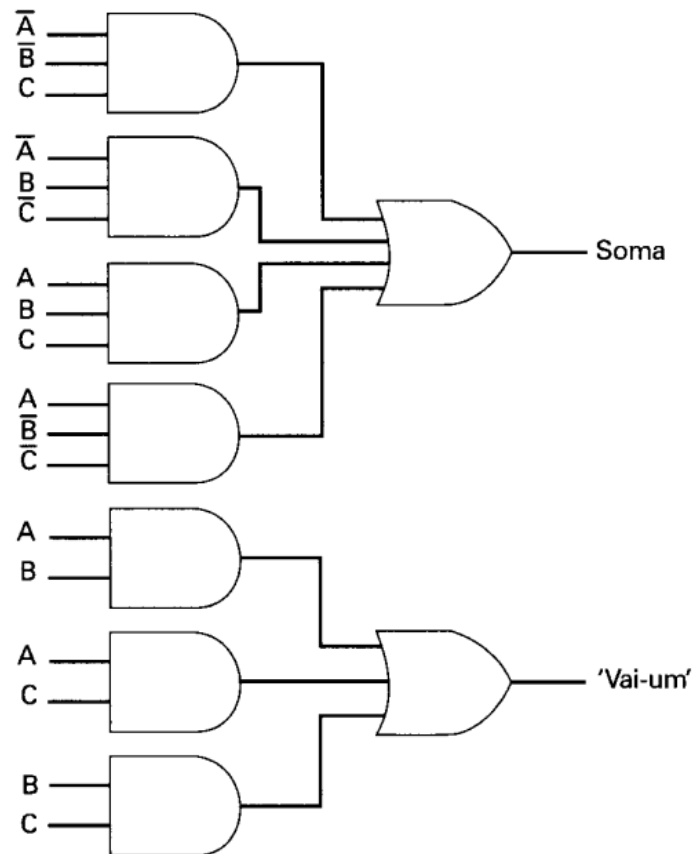
Já está simplificado!

Vai-um:

	BC			
	00	01	11	10
0			1	
1		1	1	1

$$\text{Soma} = A'BC' + AB'C' + A'B'C + ABC$$

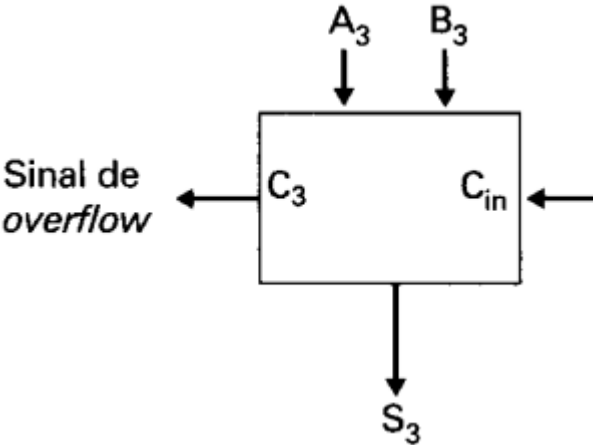
$$\text{Vai-um} = AB + AC + BC$$



Adição binária – Usando XOR

Adição de um único bit.

#	A	B	Soma	Vai-um
0	0	0	0	0
1	0	1	1	0
2	1	0	1	0
3	1	1	0	1



Adição com uma entrada de bit e ‘vai-um’.

#	C_{in}	A	B	Soma	C_{out}
0	0	0	0	0	0
1	0	0	1	1	0
2	0	1	0	1	0
3	0	1	1	0	1
4	1	0	0	1	0
5	1	0	1	0	1
6	1	1	0	0	1
7	1	1	1	1	1

$A \oplus B$



Soma = $A \oplus B \oplus C_{in}$

Vai-um = $AB + AC + BC$

Obs.: $C = C_{in}$.

Adição binária – Usando XOR

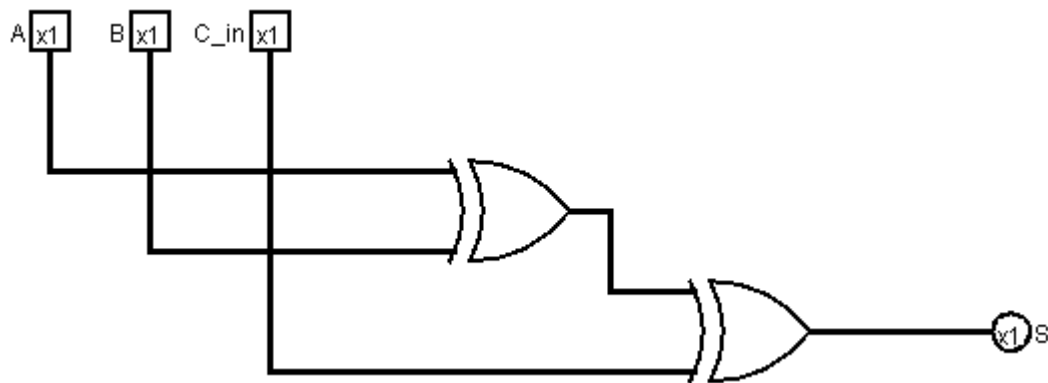
Adição com uma entrada de bit e ‘vai-um’.

#	C _{in}	A	B	Soma	C _{out}
0	0	0	0	0	0
1	0	0	1	1	0
2	0	1	0	1	0
3	0	1	1	0	1
4	1	0	0	1	0
5	1	0	1	0	1
6	1	1	0	0	1
7	1	1	1	1	1

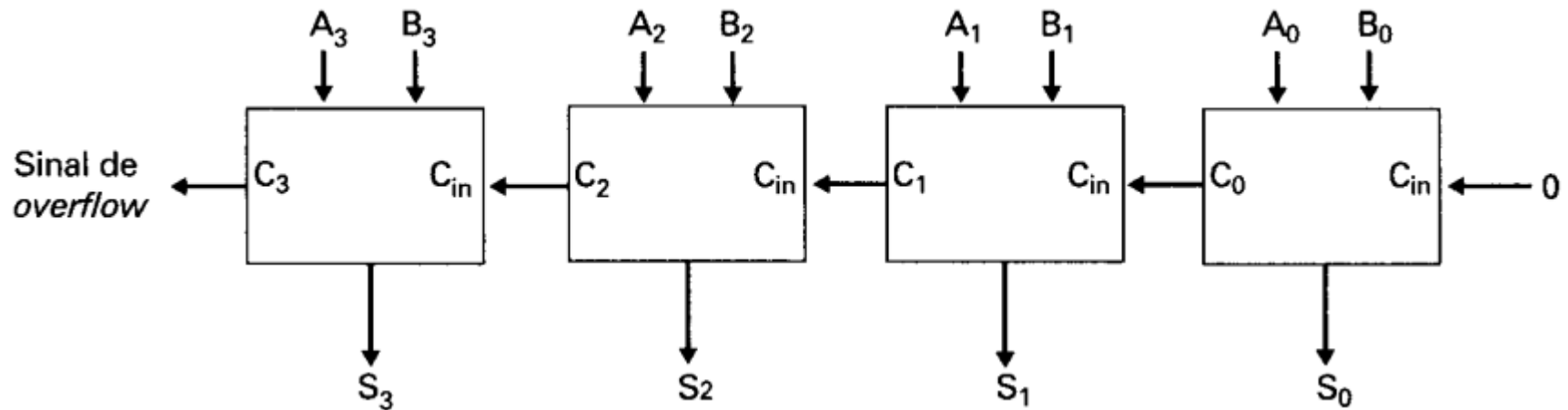
$$\text{Soma} = A \oplus B \oplus C_{in}$$

$$\text{Vai-um} = AB + AC + BC$$

$$\text{Obs.: } C = C_{in}.$$

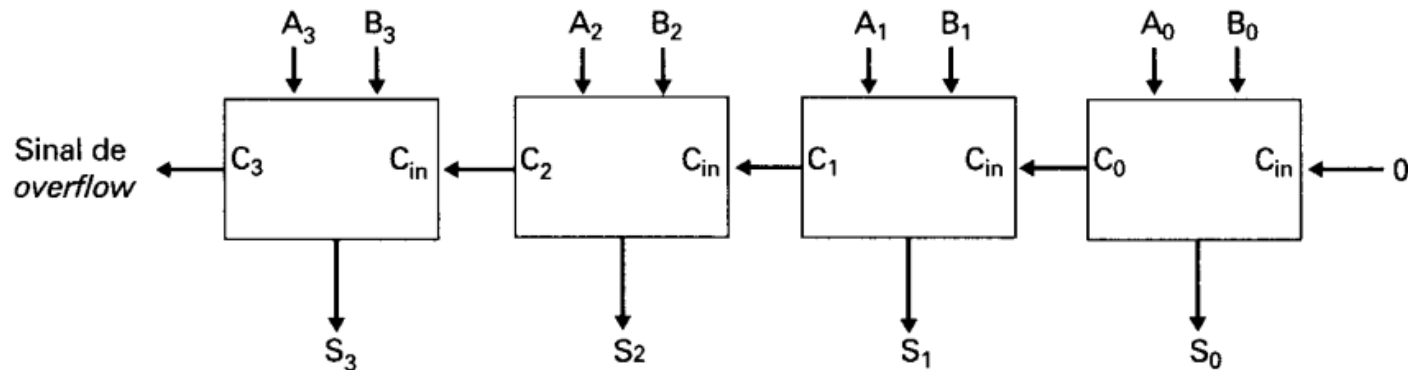


Somador de 4 bits



“Vai um” antecipado (*carry lookahead*) – 4 bits

- Cada somador depende do “vai um” do somador anterior.
 - Atraso crescente do bit menos significativo par o mais significativo.
 - Para somadores grandes o atraso acumulado é inaceitável
 - SOLUÇÃO:** Calcular o “vai um” **antecipadamente**.



$$C_0 = A_0 B_0$$

$$C_1 = A_1 B_1 + A_1 A_0 B_0 + B_1 A_0 B_0$$

$$C_2 = A_2 B_2 + A_2 A_1 B_1 + A_2 A_1 A_0 B_0 + A_2 B_1 A_0 B_0 + B_2 A_1 B_1 + B_2 A_1 A_0 B_0 + B_2 B_1 A_0 B_0$$

“Vai um” antecipado (*carry lookahead*) – 4 bits

$$C_0 = A_0B_0$$

$$C_1 = A_1B_1 + \mathbf{A_1}A_0B_0 + \mathbf{B_1}A_0B_0$$

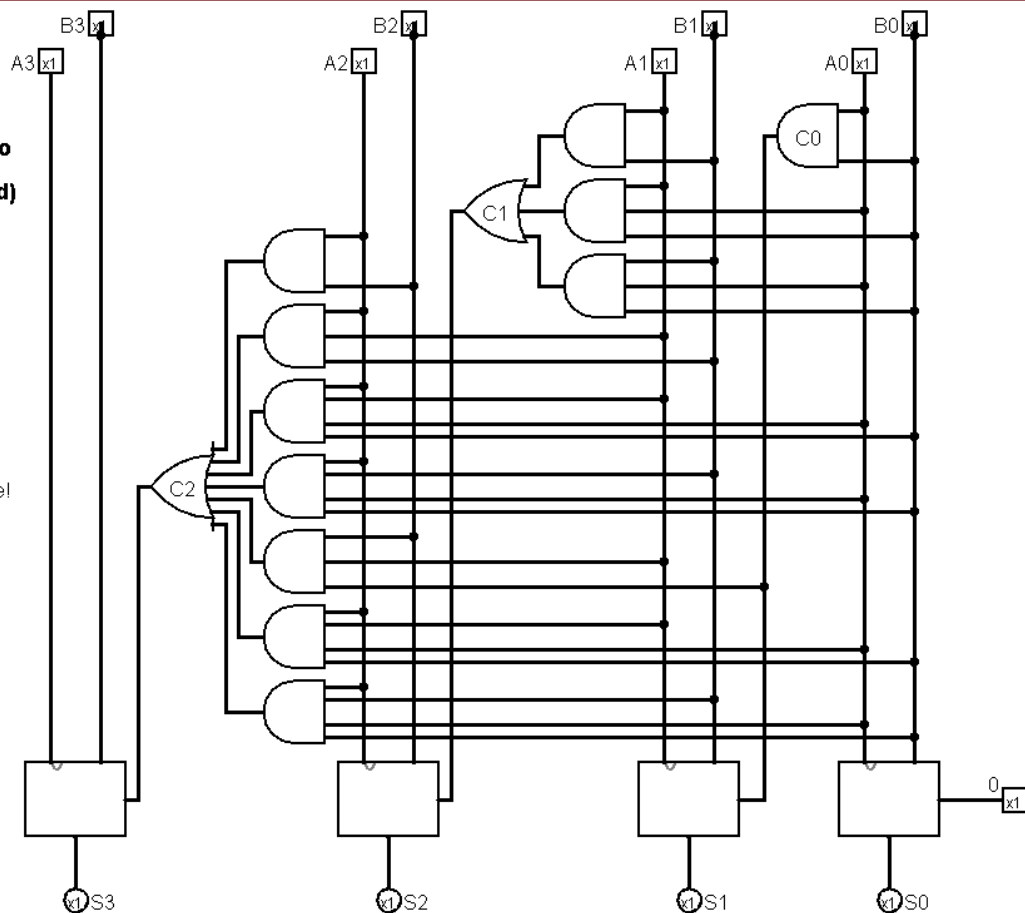
$$C_2 = A_2B_2 + \\ \mathbf{A_2}A_1B_1 + \mathbf{A_2}A_1A_0B_0 + \mathbf{A_2}B_1A_0B_0 + \\ \mathbf{B_2}A_1B_1 + \mathbf{B_2}A_1A_0B_0 + \mathbf{B_2}B_1A_0B_0$$

$$C_3 = A_3B_3 + \\ \mathbf{A_3}A_2B_2 + \mathbf{A_3}A_2A_1B_1 + \mathbf{A_3}A_2A_1A_0B_0 + \mathbf{A_3}A_2B_1A_0B_0 + \mathbf{A_3}B_2A_1B_1 + \mathbf{A_3}B_2A_1A_0B_0 + \mathbf{A_3}B_2B_1A_0B_0 + \\ \mathbf{B_3}A_2B_2 + \mathbf{B_3}A_2A_1B_1 + \mathbf{B_3}A_2A_1A_0B_0 + \mathbf{B_3}A_2B_1A_0B_0 + \mathbf{B_3}B_2A_1B_1 + \mathbf{B_3}B_2A_1A_0B_0 + \mathbf{B_3}B_2B_1A_0B_0$$

“Vai um” antecipado (*carry lookahead*) – 4 bits

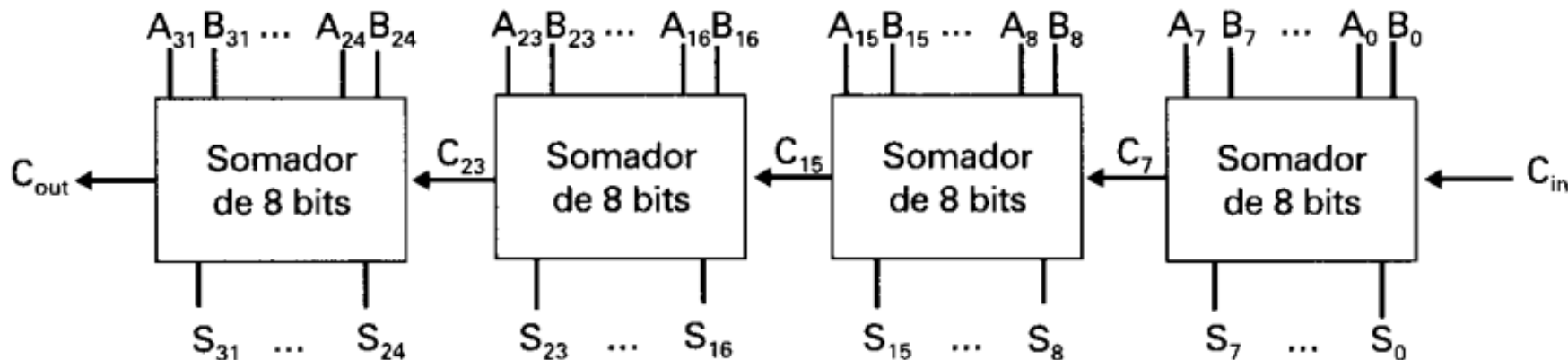
Somador de 4 bits com vai-um antecipado
(*carry lookahead*)

C3 omitido por ser muito grande!



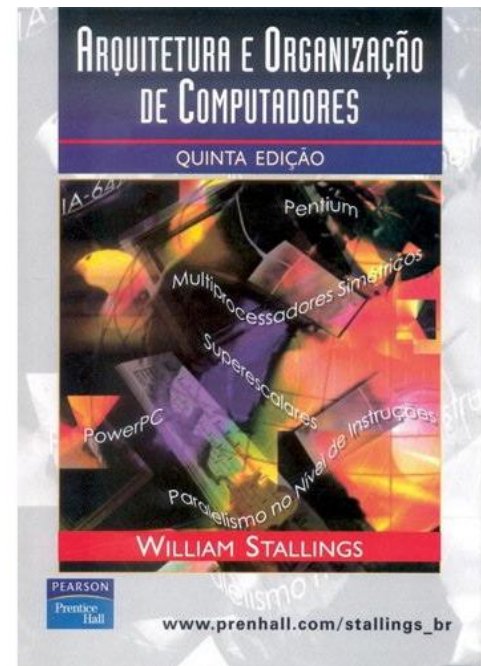
Somador de 32 bits usando somadores de 8 bits

- O “vai um” antecipado se torna complexo para somadores grandes.
 - SOLUÇÃO:**
 - “Vai-um” antecipado para blocos pequenos
 - 4 a 8 bits, no máximo.
 - Propagação do “vai-um” entre esses blocos “grandes”.



Referências

- STALLINGS, W. **Arquitetura e Organização de Computadores**, 5. Ed., Pearson, 2010.
 - Apêndice A
- **Recursos do livro:**
 - <http://williamstallings.com/ComputerOrganization/styled-2/>
 - Em Online Chapter, baixar:
 - 20-DigitalLogic.pdf



Material complementar

- How Computers Calculate - the ALU: Crash Course Computer Science #5
 - <https://www.youtube.com/watch?v=1I5ZMmrOfnA>

FIM